Peak and Trend at the Edge: A Fog-to-Cloud Monitor for a Two-Tower Elevator and Escalator Fleet

Rasool Basha Durbesula
Student ID: X24205478
MSc in Cloud Computing
Fog and Edge Computing (H9FECC)
National College of Ireland
x24205478@student.ncirl.ie

***Abstract*—A lift can fail in two very different ways, and a monitor that treats them alike will miss one of them. Some faults arrive as a slow drift: a motor that runs a little hotter each hour, a ride that roughens as a guide wears, a car that settles into a sluggish crawl. Others arrive in a single instant: one overweight trip that a whole window of ordinary loads would average away. This project is built around that distinction. Ten sensor processes drive five signals, motor temperature, door cycles, cab vibration, load weight, and travel speed, across two elevator and escalator towers; a fog node on the edge host windows each tower-and-signal stream and reduces it to a five-number summary of count, minimum, maximum, average, and latest value. Each alarm rule then reads the field of that summary its fault actually lives in: the window average for the three signals that fail by drifting, and the window maximum for load weight, where a single peak is the hazard and an average would hide it. Door cycles are carried as context and raise no alarm at all. A serverless backend on Amazon SQS, AWS Lambda, and Amazon DynamoDB persists every window, and a dashboard served through Amazon API Gateway shows each tower as nominal or alerting from those stored windows. The system was provisioned to a real Amazon Web Services account rather than a local emulator; 122 automated tests pass, and verification against the live endpoint confirmed both towers streaming all five signals, a ride-quality fault and an overload warning firing together on one tower, live trend charts, and a healthy pipeline, with the frontend delivered from Amazon S3.**

***Keywords*—elevator monitoring, condition monitoring, predictive maintenance, vibration analysis, fog computing, edge computing, Internet of Things, serverless computing, Amazon SQS, AWS Lambda, Amazon DynamoDB**

I. Introduction

Vertical transport is one of the few pieces of building machinery that carries people directly, so an elevator or escalator that misbehaves is a safety matter before it is a maintenance one. The signals that reveal how a car is faring are well understood in condition-based maintenance: a motor's temperature tracks how hard it is working and how well it is cooled, vibration in the cab is the classic early trace of wear in the guides and bearings, travel speed shows whether the drive is holding its commanded profile, and load weight bounds what the car is being asked to lift [1], [2]. Door cycles count how much service the car has given, which frames everything else but is not itself a fault. Reading these signals continuously across a fleet is what separates a planned repair from an unplanned entrapment.

The distributed-systems question is where those readings are turned into a decision. The Internet of Things makes it cheap to instrument every car in a building [3], but streaming every raw sample from every tower to a distant cloud region to be judged there is wasteful in two directions at once. It spends bandwidth continuously on readings that, taken singly, rarely change anything, and it defers the one moment that matters, a signal crossing into its danger range, until after a network round trip. Fog computing was proposed to close that gap by placing a compute tier between the sensors and the cloud, near enough to the sources to summarise, filter, and evaluate rules before anything travels [4], [5], and it is one expression of the broader movement of computation back toward the network edge [6], [7]. Behind that edge tier, the cloud supplies the elastic, pay-as-used model that removes the need to size a server for a peak that sits idle most of the day [8].

This project builds that two-tier design for a scaled but faithful slice of a serviced fleet: two towers, designated tower-a and tower-b, each carrying the same five signals, run as ten independent sensor processes. What gives the design its own character is how it decides that a window is dangerous. When each window closes it is reduced to a five-number summary, and every alarm rule is bound not just to a signal but to the one field of that summary where its fault shows itself. A motor overheating, a ride roughening, and a car stalling are all trends, so they are judged on the window average; an overload is a single event, so it is judged on the window maximum, because averaging a brief overweight peak across ten seconds of normal trips would erase exactly the reading that matters. That deliberate pairing of fault to statistic is the report's recurring theme.

The work pursues four objectives, each demonstrated on a running system rather than only described. The first is a sensor and fog layer that genuinely windows and evaluates five heterogeneous signals at sampling and dispatch rates that are configurable rather than fixed in code. The second is a backend

that scales through managed, elastic AWS primitives instead of a single always-on server. The third is a dashboard that turns stored windows into a fleet reading an operator can act on, marking each tower nominal or alerting. The fourth, and the one that separates a working system from a plausible one, is deployment onto a real public-cloud account and verification against the live endpoint. Scope is held to these layers at a two-tower scale rather than a whole building's worth of cars. The remainder of this paper is organised as follows. Section II presents the architecture and the reasoning behind each choice, including the alternatives weighed and the security posture. Section III covers the implementation, the automated tests, continuous integration, the real deployment, and the evidence gathered from the running system. Section IV closes with a critical assessment of the trade-offs, the limits, and where the work could go next.

II. System Architecture and Design

Fig. 1 shows the deployed system end to end: ten sensor processes and the fog node sharing one edge host, a managed queue, two independently deployed functions, a key-value store, and a browser-facing bucket reached through a managed gateway.

A. Sensor and Fog Layer

Each of the ten sensor processes represents one signal on one tower. On every sampling tick a process advances a bounded random walk: it nudges its previous value by a step drawn uniformly from a small symmetric range and holds the result inside a physically plausible band for that signal, so consecutive readings stay close together like a real transducer rather than independent noise. The step sizes are set per signal to match how the real quantity moves, so cab vibration and load weight, which can change abruptly, swing further between ticks than motor temperature, which drifts slowly. Sampling and dispatch run on two independent timing loops, and each loop is drift corrected: rather than sleep for a fixed interval after each tick, which lets the tick's own work push the next one steadily later, the loop schedules each tick against the next ideal boundary computed from a fixed origin, so any lateness in one tick is absorbed by the next rather than accumulating over an hour of operation. Regular spacing matters here because the fog groups readings into fixed windows, and a sampler that quietly slipped out of step would bias every average it feeds. The sampling interval defaults to two seconds and the dispatch interval to ten, and keeping them separate lets a fast-moving quantity be sampled often while still being shipped in economical batches. Each dispatch is a compact JSON object naming the signal, the tower, and the unit and carrying the batch of values gathered since the last dispatch, which at the default rates is roughly five readings amounting to a few hundred bytes on the wire, well under a kilobyte. The process posts it over HTTP to the fog node's ingest endpoint, and if that post fails because the fog node is briefly unreachable, the batch is placed back at the front of the process's own outbox and retried on the next dispatch, ahead of anything sampled in the meantime, so a transient outage delays readings in arrival order instead of dropping or reordering them.

The fog node is a Node.js service built directly on the runtime's own HTTP server, running as its own container on the edge host distinct from the sensor processes and the cloud functions. Requests are dispatched by a small middleware router: each route is an ordered chain, and a stage either answers the request or calls the next stage, so ingest validation and ingest handling are two separate stages of the same route rather than one tangled function. The validation stage parses the body, rejects a batch that is missing a field, carries a non-numeric value, or exceeds a size cap with a clear error and its own status code, and only then passes control on; nothing malformed ever reaches the buffer. The handler stage records each reading into a run ledger, a map keyed on the tower-and-signal pair, and separately notes each signal's unit, since the wire carries a unit but the ledger key does not. Because the runtime is single threaded, the ledger needs no lock: when the window boundary arrives, the ledger is drained and emptied in one call, handing back every non-empty group and leaving a fresh empty map in place for the next window, and the summarising happens on the returned snapshot outside that step.

When a window closes, each non-empty tower-and-signal group is reduced to a summary carrying the count, minimum, maximum, average, and latest value, and that summary, not any raw reading, is evaluated against the alarm rules. The rules are the heart of the design. Each is a predicate bound to one signal and, implicitly, to one field of the summary. An average motor temperature above eighty-five degrees raises an overheating alarm; an average cab vibration above six millimetres raises a ride-quality fault; an average travel speed below half a metre per second raises a stall warning; and a maximum load weight above one thousand kilograms raises an overload warning. Three of the four read the window average, because a motor that is genuinely overheating, a ride that is genuinely rough, and a car that is genuinely crawling all show up as a sustained shift the average captures. The fourth reads the window maximum on purpose, because an overload is a single overweight trip and not a sustained condition, and the average of one heavy trip among nine ordinary ones would sit comfortably below the limit while the car was actually overloaded. Door cycles carry no rule; the count is stored and shown as service context but is never a hazard in itself. The fog node also publishes this rule set, as a descriptive table naming each signal's field, operator, and limit, on a read-only endpoint, so the dashboard can show operators the thresholds in force without hard-coding them a second time.

The fog node's output interface is a real managed queue rather than a direct write to storage. On each window boundary it batches the closed summaries and sends them to Amazon SQS in grouped calls, each carrying up to the service's per-call limit of ten summaries, so a whole flush costs a number of send operations proportional to the summaries closing divided by ten rather than one send per summary. The queue's address

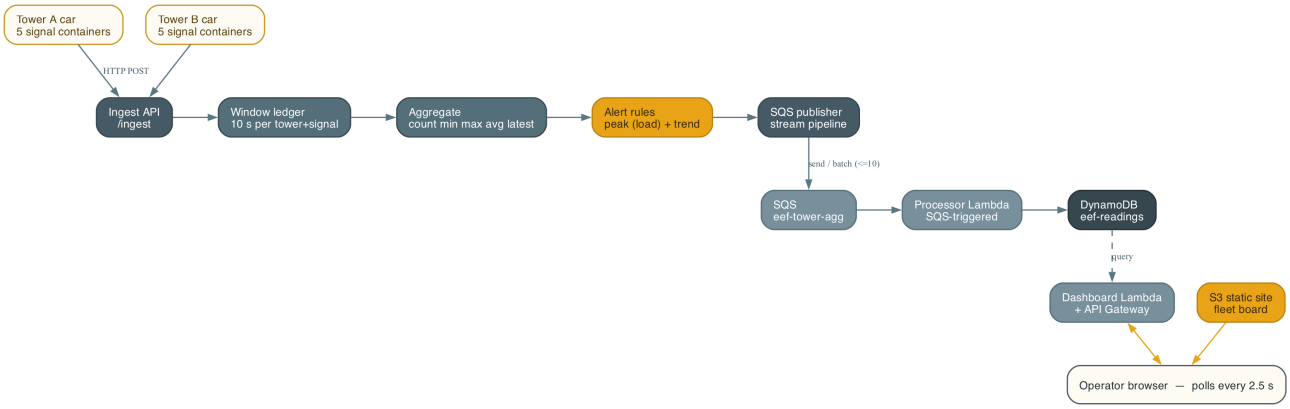

Fig. 1. Deployment topology. Ten sensor processes, two towers of five signals each, and the fog node share one Amazon EC2 edge host. The ingest path runs from the fog node through Amazon SQS and the processor function into the Amazon DynamoDB table; the serving path runs from the dashboard function's read of the stored windows out through Amazon API Gateway to the browser, with static assets served from Amazon S3. The fog node reduces each window to a five-number summary and applies the peak and trend rules before anything is sent. The dashed edge is the dashboard function's query back over the stored windows.

is resolved once and remembered rather than looked up on every publish, and because a fresh deployment may still be provisioning the queue when the edge host boots, the lookup retries under an exponential backoff that starts at a fifth of a second, doubles up to a three-second ceiling, and gives up only after many attempts. A send that still fails is logged and the cycle continues rather than crashing the node, a bias toward keeping the edge live that suits a monitoring workload where the next window is only seconds away. The publisher is written as a genuine stream: a transform feeding a writable sink that performs the send, with each single publish tracked to its own completion, and a batched path used at flush that chunks the closed summaries against the ten-entry limit.

B. Backend Layer

The cloud tier follows a single ingest path with a branch for the operator interface. A queue receives the fog node's summaries, one function drains the queue and writes to storage, and a second, separately deployed function reads that storage back to answer the dashboard. Splitting the write path from the read path is deliberate: ingestion and querying then scale and fail independently, so a surge of windows cannot slow the dashboard and a burst of operator activity cannot back up ingestion.

Queue instead of a direct call. The fog node could invoke the storage function directly and synchronously, dropping the queue and saving a hop. That option was declined. A synchronous call binds the edge to the availability and latency of the cloud function: throttle it, leave it cold, or take it down briefly, and the fog node stalls or sheds data as the back-pressure travels back toward the sensors. A queue severs that coupling, absorbs bursts, retries on the consumer's behalf, and lets the fog node treat a summary as delivered the moment it is enqueued [9]. On a ten-second cadence the small added latency is immaterial, while the decoupling and durability are exactly what an intermittent edge link needs.

Event-driven functions instead of a standing server. Both the ingestion and the interface tiers run as event-driven functions. This fits work that arrives in bursts and, for the interface, only intermittently: billing is per request rather than for idle capacity, and the platform adds and removes instances as demand shifts, with nothing reserved in advance [8]. The recognised cost is the cold start after a period of idleness [10]; its effect here is small, because steady queue traffic keeps the ingestion function warm and the dashboard's own polling keeps the interface function warm. The processor function is triggered by the queue, holds no state between invocations, and is billed only for the work it does.

Key-value store instead of a relational database. A relational engine would offer joins and open-ended queries, neither of which this workload uses. Its access pattern is narrow and known in advance: append summaries in time order, then fetch the most recent windows for a given signal. Amazon DynamoDB, an on-demand key-value store built for that mix of heavy writes and key-based reads, holds its performance as the table grows and spares the operational overhead of running a relational server, the balance the original Dynamo work set out and the managed service now carries [11], [12]. The key design follows straight from the query. Each stored window is keyed on its signal as the partition key and on a sort value that leads with the window's closing timestamp and then the tower identifier, so windows for one signal are already ordered chronologically inside a single partition and the newest are read with one bounded, descending, single-partition query rather than a scan; the tower in the sort tail keeps the two towers' windows for the same signal from colliding.

C. Serving Layer, the Fleet Board, and Security

The dashboard reaches the read function through a managed gateway that publishes a public REST endpoint. Object storage holds the static frontend, one page, a stylesheet, the page script, and a charting library kept in the repository rather than pulled from a content-delivery network. Inside the deployed

function the same middleware-router pattern used on the edge dispatches requests, and it exposes a small, read-only surface: a readings endpoint returning a recent series for one signal, a towers endpoint giving each tower's latest window per signal together with whether it is nominal, a per-tower variant of that view, a thresholds endpoint that forwards the fog node's published rule set, a backend-statistics endpoint, and a health endpoint assembled from live sources, the store for the freshest window, the queue, the processor function's deployment state, and the fog node's reachability, returning a pipeline flag that is true only when a window actually reached the store recently.

The towers view is where the stored windows become a fleet reading. For each tower the read function collects the latest window of every signal and gathers any alarm keys those windows carry, and a tower is marked nominal only when none of its latest windows is alarming; otherwise its alarms are listed so the frontend can flag it. Because the fog node has already evaluated the peak and trend rules and stored the resulting alarm keys on each window, the dashboard does not re-derive them, it simply reports what the edge already decided, which keeps a single definition of what counts as a fault. On the page, each tower is a card listing its five signals with the current value and a small bar for each, a fault-bearing signal picked out in the alert colour, and a status pill reading nominal or alerting; a row of window-average trend charts tracks each signal across both towers, and a banner names any alarm currently firing. A fixed browser-side timer re-fetches the towers view, the health, and the statistics every 2.5 seconds and repaints from whatever the responses hold, so the board stays current without a manual reload and without any server-push channel. The page learns which gateway origin to call from a single placeholder written into it and substituted with the real endpoint when the assets are uploaded; run locally, where one process serves both the page and the interface, the placeholder resolves to empty and requests fall back to the same origin. The security posture is modest and deliberate for a scaled demonstration: the fog node's health and threshold endpoints answer without a credential check because they expose only aggregate window state and configured limits, never a live per-sensor reading, while the cloud tier relies on the managed services' own access control and on scoping each function to the queue, table, and gateway it needs.

III. Implementation

The sensor processes, the fog node, and both cloud functions are written in JavaScript on the Node.js runtime. The fog node and the dashboard's local server are built directly on the runtime's HTTP server with a hand-written middleware router, so neither carries a web framework; the AWS SDK for JavaScript carries the queue, table, and function calls, and the frontend renders its trends with a locally vendored charting library so the page has no external dependency at load time. The pieces are kept as small single-purpose modules, the drift-corrected timing separated from the sampling, the run ledger separated from the aggregation, and the alarm rules separated from the publisher, so each is unit-testable on its own without standing up a server or reaching into module-scope state. The edge tier is orchestrated with Docker Compose, the cloud tier is provisioned with Terraform, and continuous integration runs on GitHub Actions.

A. Testing and Continuous Integration

The system carries an automated suite of 122 tests spread across every component: fourteen over the sensor's bounded walk, its drift-corrected cadence, and its retain-in-order dispatch; fifty-seven over the fog node's router, ingest validation, run ledger, window aggregation, alarm rules, and streaming publisher; ten over the processor's transform into a stored item; and forty-one over the dashboard's towers assembly, health, thresholds proxy, and request routing. They run without any cloud connection by exercising the units against hand-written fake AWS clients, so each test can assert on the exact shape of the request a given call would issue rather than depending on a live service. The alarm-rule tests are pointed deliberately at the boundaries, checking a value one step inside a threshold and one step outside it, and the overload rule is tested specifically to confirm it fires on a single peak that the window average leaves below the limit, which is the whole reason it reads the maximum rather than the mean. The publisher tests confirm that a flush is split into batches no larger than the queue's per-call limit and that the stream path tracks each publish to its own outcome, the processor tests check the composite sort value that orders windows within a partition, and the fog ingest validation is proven with a real HTTP request against a running local server rather than only a unit test of the validator in isolation, since that request path is what the deployment actually exercises. The suite runs on every push through GitHub Actions, so a change that breaks a threshold, a batch size, a query shape, or the nominal decision fails in continuous integration rather than after deployment.

Testing against fakes rather than a live queue was a deliberate choice with a cost worth acknowledging: a fake asserts on the request a call would make, not on how the managed service behaves under load, so the batched-send limit and the queue's own retry semantics were reasoned about from the service documentation [13] and then confirmed against the live deployment rather than in unit tests.

B. Deployment and Live Verification

The cloud tier was provisioned to a real AWS account in a single Terraform apply that created twenty-four resources, with the account confirmed before anything was applied and the working state isolated so the apply could add only its own resources and never disturb anything already present. Describing the whole cloud tier as code, rather than clicking it together once, means the same definition builds the queue, both functions with their triggers and permissions [14], the table, the gateway with its single catch-all route [15], and the two buckets in one reproducible step, and tears them down as cleanly. The edge host runs the fog node and the ten sensor containers under Docker Compose; all the deployed resources carry a common prefix so the deployment is easy to identify

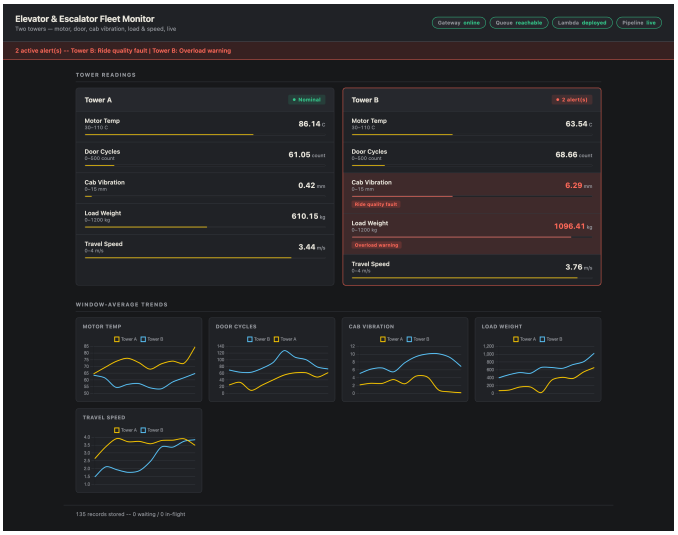

Fig. 2. The deployed dashboard reading from the live endpoint. Each tower shows its five signals with a current value and bar and a nominal or alerting pill; the row of window-average trend charts tracks each signal across both towers; the banner names the alarms currently firing; and the header health strip shows the four pipeline indicators. Here one tower is alerting on a ride-quality fault and an overload warning at once.

and to remove as a unit, and a fixed public address in front of the edge host lets the read function reach the fog node's health and threshold endpoints at the same location across restarts.

Verification was carried out against the live deployment rather than a local emulator, and Fig. 2 is the deployed dashboard during that check. Within about a minute of start-up the pipeline reported healthy end to end: the fog node, the queue, the processor function, and the freshest-window recency all read healthy, and the newest window was a few seconds old on each poll. Both towers rendered all five signals with their current value and bar, and the peak and trend rules were visible working together: on one tower the cab vibration crossed its average limit and the load weight crossed its maximum limit in the same window, so the ride-quality fault and the overload warning both fired, the banner named them, and the tower's pill flipped to alerting while the other tower held nominal. The backend-statistics endpoint showed the archived record count climbing across successive polls rather than a static number, which confirms the ingestion function was draining the queue and writing on an ongoing basis, and reaching the dashboard from a browser also exercised the cross-origin path from the storage bucket to the gateway, which returned data cleanly.

IV. Conclusion

This project set out to build and, above all, to deploy a fog-to-cloud pipeline that turns five signals across two elevator and escalator towers into a live fleet reading, and it met that goal on a running system rather than on paper. The edge tier does genuine work before anything travels: it windows and reduces each tower-and-signal stream, applies its alarm rules locally, and ships only compact summaries, so a fault is judged in the window it appears rather than after a trip to a distant region. The cloud tier scales through decoupling and elasticity rather than a pre-sized server, using a queue to absorb bursts and separate the write and read paths, event-driven functions billed only for the work they do, and an on-demand key-value store whose key design turns the dashboard's one query into a single-partition read. The whole system was provisioned to a real account by one infrastructure-as-code apply and verified end to end against the live endpoint, with 122 automated tests guarding its behaviour and continuous integration running them on every change.

The choice that shaped the system most was refusing to treat the five signals uniformly at the moment of judgement. Reducing a window to a five-number summary is ordinary; deciding, per signal, which of those numbers the fault actually lives in is what made the monitor honest. Binding overheating, rough riding, and stalling to the window average and binding overload to the window maximum meant that a slow drift and a single dangerous instant were both caught by the statistic that exposes them, where a single blanket rule would have let the overload hide inside its own average. The supporting choice was quieter but load-bearing: because the windows those rules read are only as trustworthy as their timing, the samplers were made drift correcting, so every average compares a like span of time and the peak that trips an overload is a real peak rather than an artefact of a sampler that had slipped out of step.

The work has clear limits. It runs at a two-tower scale with simulated sensors, its thresholds are fixed in code rather than learned from each car's own baseline or adjustable at runtime, and its security posture is intentionally modest for a demonstration. Natural next steps are to widen it to a real fleet of instrumented cars and real transducer feeds, to move the fixed thresholds to a managed configuration a maintenance team could tune per car without a redeploy, to add authentication and a custom domain in front of the public endpoint, and to retain longer history so a car's slow slide toward its limits can be trended for genuine predictive maintenance rather than only judged on the latest window. None of these changes the shape of the pipeline, which is the point: the edge-aggregation and cloud-serving design carries from two towers to many without rethinking the tiers.

References

[1] R. B. Randall, *Vibration-based Condition Monitoring: Industrial, Aerospace and Automotive Applications*. Chichester, U.K.: Wiley, 2011.

[2] A. K. S. Jardine, D. Lin, and D. Banjevic, "A Review on Machinery Diagnostics and Prognostics Implementing Condition-Based Maintenance," *Mechanical Systems and Signal Processing*, vol. 20, no. 7, pp. 1483–1510, 2006.

[3] L. Atzori, A. Iera, and G. Morabito, "The Internet of Things: A Survey," *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010.

[4] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog Computing and Its Role in the Internet of Things," in *Proc. MCC Workshop on Mobile Cloud Computing*, 2012, pp. 13–16.

[5] A. Yousefpour et al., "All One Needs to Know about Fog Computing and Related Edge Computing Paradigms: A Complete Survey," *Journal of Systems Architecture*, vol. 98, pp. 289–330, 2019.

[6] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge Computing: Vision and Challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

[7] M. Satyanarayanan, "The Emergence of Edge Computing," *Computer*, vol. 50, no. 1, pp. 30–39, 2017.

[8] M. Armbrust et al., "A View of Cloud Computing," *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.
[9] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Boston, MA: Addison-Wesley, 2003.
[10] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, "Peeking Behind the Curtains of Serverless Platforms," in *USENIX Annual Technical Conference*, 2018, pp. 133–146.
[11] G. DeCandia et al., "Dynamo: Amazon's Highly Available Key-value Store," in *Proc. ACM SIGOPS Symp. Operating Systems Principles*, 2007, pp. 205–220.
[12] M. Elhemali et al., "Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service," in *USENIX Annual Technical Conference*, 2022, pp. 1037–1048.
[13] Amazon Web Services, "Amazon Simple Queue Service Developer Guide," 2024. [Online]. Available: https://docs.aws.amazon.com/sqs/
[14] Amazon Web Services, "AWS Lambda Developer Guide," 2024. [Online]. Available: https://docs.aws.amazon.com/lambda/
[15] Amazon Web Services, "Amazon API Gateway Developer Guide," 2024. [Online]. Available: https://docs.aws.amazon.com/apigateway/
[16] IEEE, "Manuscript Templates for Conference Proceedings," 2024. [Online]. Available: https://www.ieee.org/conferences/publishing/templates.html

The source code and deployment configuration are available at [GitHub repository URL].